ARCHIVER MES FICHIERS

```
tar -cvf nom_archive.tar nom_dossier/
```

J'utilise trois options :

- `-c` : signifie créer une archive `tar` ;
- `-v` : signifie afficher le détail des opérations ;
- `-f` : signifie assembler l'archive dans un fichier.

Essayons de faire cela sur notre dossier `tutoriels` :

```
$ tar -cvf tutoriels.tar tutoriels/
tutoriels/
tutoriels/les-principaux-widgets.tuto
tutoriels/la-surcharge-d-operateurs.tuto
tutoriels/l-heritage.tuto
```

Ici on archive le dossier `tutoriels` et donc son contenu. Grâce à `-v`, on voit bien la liste des fichiers qui ont été archivés.

```
tar -cvf archive.tar fichier1 fichier2 fichier3
```

C'est possible et ça fonctionne. Toutefois, il est de coutume sous Linux de placer **d'abord** les fichiers dans un dossier avant de les mettre dans le `tar`. Cela permet d'éviter, lorsqu'on extrait les fichiers de l'archive, que ceux-ci aillent se mêler à d'autres fichiers. Nous allons voir ce problème maintenant.

`-tf` : afficher le contenu de l'archive sans l'extraire

Vous venez de recevoir une archive `tar` qu'on vous a envoyée. Bien. Mais que contient-elle ?
Avant d'extraire quoi que ce soit, vous aimeriez peut-être voir son contenu. C'est possible avec `-tf` :

```
$ tar -tf tutoriels.tar
tutoriels/
tutoriels/les-principaux-widgets.tuto
tutoriels/la-surcharge-d-operateurs.tuto
tutoriels/l-heritage.tuto
```

`-rvf` : ajouter un fichier

Si vous avez oublié un fichier, vous pouvez toujours l'ajouter par la suite avec `-rvf` :

```
$ tar -rvf tutoriels.tar fichier_supplementaire.tuto
tutoriels/fichier_supplementaire.tuto
```

`-xvf` : extraire les fichiers de l'archive

Pour extraire les fichiers, on va utiliser les options `-xvf` (`-x` pour *eXtract*) :

```
$ tar -xvf tutoriels.tar
tutoriels/
tutoriels/les-principaux-widgets.tuto
tutoriels/la-surcharge-d-operateurs.tuto
tutoriels/l-heritage.tuto
```

gzip & bzip2 : compresser une archive

Vous avez maintenant créé une belle archive `tar`. Tous vos fichiers sont réunis là-dedans. Voyons comment compresser cela.

Nous disposons de deux programmes de compression bien répandus dans le monde Linux :

- `gzip` : c'est le plus connu et le plus utilisé ;
- `bzip2` : il est un peu moins fréquemment utilisé. Il compresse mieux mais plus lentement que `gzip`.

`gzip` : la compression la plus courante

Concrètement, le programme `gzip` s'utilise de la manière la plus simple qui soit :

```
gzip tutoriels.tar
```

L'archive est compressée et gagne ensuite le suffixe `.gz` . Elle s'appelle donc désormais `tutoriels.tar.gz` . Voilà pourquoi vous voyez circuler sur l'internet des fichiers `.tar.gz` : cela signifie que ce sont des archives compressées !

Pour décompresser l'archive ensuite, il suffit d'utiliser `gunzip` :

```
gunzip tutoriels.tar.gz
```

L'archive retrouve son état non compressé en `.tar` . Vous pouvez maintenant extraire les fichiers de l'archive comme vous avez appris à le faire un peu plus tôt avec `tar -xvf` . ;-)

`bzip2` : la compression la plus puissante

Le fonctionnement de `bzip2` est le même que celui de `gzip` :

```
bzip2 tutoriels.tar
```

Une archive compressée `tutoriels.tar.bz2` sera alors créée. Pour la décompresser, utilisez `bunzip2` :

```
bunzip2 tutoriels.tar.bz2
```

Vous retrouvez un `.tar` que vous pouvez extraire avec `tar -xvf` .

ARCHIVER ET COMPRESSER EN MÊME TEMPS AVEC TAR

`-zcvf` : archiver et compresser en `gzip`

Vous connaissez `tar -cvf` pour créer une archive `tar`. Si vous rajoutez l'option `-z`, l'archive sera automatiquement compressée avec `gzip`.

```
tar -zcvf tutoriels.tar.gz tutoriels/
```

Voilà comment on obtient une archive compressée en une seule commande. :-)

Pour décompresser, c'est pareil, sauf que le `-c` est remplacé par un `-x` comme tout à l'heure :

```
tar -zxvf tutoriels.tar.gz
```

ARCHIVER ET COMPRESSER EN MÊME TEMPS AVEC TAR

`-jcvf` : archiver et compresser en `bzip2`

Le principe est le même avec `-j` à la place de `-z` :

```
tar -jcvf tutoriels.tar.bz2 tutoriels/
```

Et pour extraire :

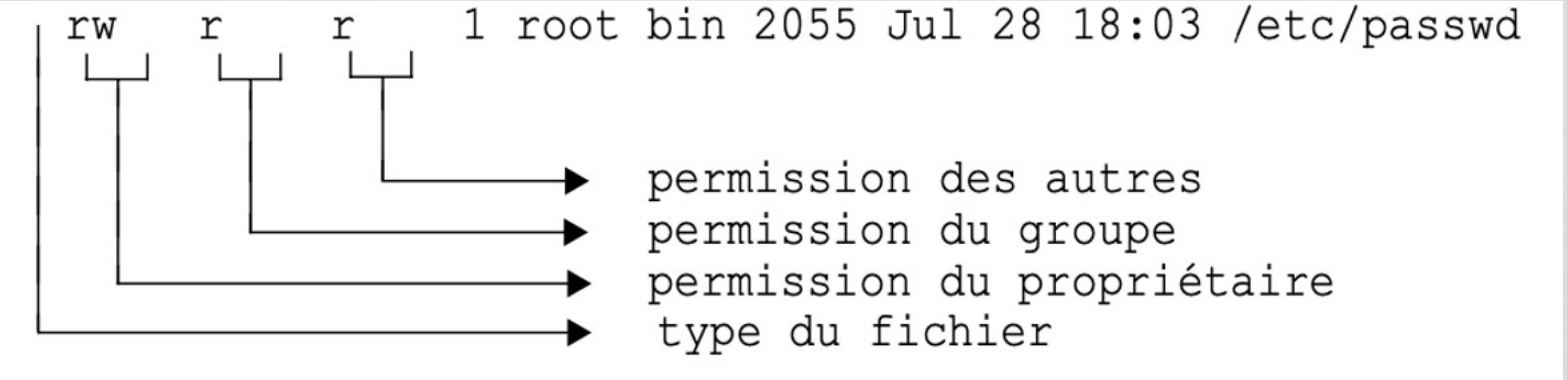
```
tar -jxvf tutoriels.tar.bz2 tutoriels/
```

PROTECTION DES FICHIERS

DROIT D'ACCÈS AUX FICHIERS

- À chaque fichier est associé un ensemble d'indicateurs précisant les droits d'accès au fichier.
- Pour chaque fichier il existe trois types d'utilisateurs :
- le propriétaire du fichier,
- les membres du groupe propriétaire du fichier,
- les autres utilisateurs du système.
- Pour chaque fichier et par type d'utilisateur il existe trois modes principaux :
  - autorisation d'écriture (w)
  - autorisation de lecture (r)
  - autorisation d'exécution (x)

- Ainsi, à chaque fichier Linux sont associés 10 attributs (1 pour désigner le type, et 9 attributs de protection, 3 pour le propriétaire, 3 pour le groupe et 3 pour les autres utilisateurs). L'ensemble des renseignements sur un fichier est obtenu en utilisant la commande ls -l.



MODIFICATION DES DROITS D'ACCÈS

- La protection d'un fichier ne peut être modifiée que par le propriétaire à l'aide de la commande **chmod** (CHange MODe). Il existe deux modes d'utilisation de cette commande. La première (la plus ancienne) utilise la description des protections par un nombre octal.

EXEMPLE

- **rwx rw- r-x** est représenté par le nombre octal **765**. En effet, une lettre équivaut à 1, un tiret (espace) à 0.
  On a donc rwx rw- r-x=111 110 101=765

- Cet exemple va modifier la protection du fichier toto de la manière suivante :
  r x rw rwx autorisant un accès lecture et exécution au propriétaire, lecture et écriture au groupe et un accès sans restriction aux autres (cas très rare).

- chmod 567 toto

- Le deuxième mode d'utilisation de chmod, le mode symbolique, permet une description absolue ou relative des droits d'accès, comme suit :
- chmod [who] [op] [permission] fichier
- **[who]** est une combinaison de lettre u (user=propriétaire), g (groupe), o (other=autre) ou a (all=tous) pour ugo,
- **[op]** + permet d'ajouter un droit d'accès, - de supprimer un droit d'accès et = d'affecter un droit de manière absolue (tous les autres bits sont remis à zéro),
- **[permission]** r (read=lecture), w (write=écriture), x (exécution).

EXEMPLE

- **chmod u w file** supprime le droit d'écriture au propriétaire.
- **chmod g+r file** ajoute le droit de lecture pour le groupe.
- **chmod ug=x file** accès uniquement en exécution pour le propriétaire et le groupe, pas de modification pour les autres.

DROIT D'ACCÈS À LA CRÉATION DU FICHIER

- La protection d'un fichier, ainsi que le nom du propriétaire (le vôtre) et le nom du groupe auquel vous appartenez, sont établis à sa création et ne peuvent être modifiés que par son propriétaire.
- La commande umask permet de définir un masque de protection des fichiers (et répertoires) lors de leur création. Cette commande se trouve en général dans le fichier .bashrc, mais elle peut être exécutée à tout moment. Le masque est exprimé en base 8.

EXEMPLES

```
umask 022
```

La valeur 022 est soustraite de la permission permanente (111 111 111) :

```
111 111 111   <= permission permanente
000 010 010   <= on enlève les bits dont on ne veut pas
111 101 101    => 755
```

umask 022 permet de créer des fichiers répertoires dont la protection est rwx r x r x. C'est souvent l'option par défaut.

Attention

Pour les fichiers ordinaires, *umask 022* donnera une protection de type rw r r , car **la possibilité d'exécution n'est pas autorisée sur les fichiers ordinaires**.

Voici donc ce qui se passe après avoir lancé *umask 022* :

```
touch f1
ls -l f1
rw  r   r    1  xstra staff  40 jan 15 16:04  f1
```

- La forme symbolique de la commande umask est beaucoup plus agréable à l'utilisation que la forme octale. Acceptée par le bash et le Z-shell (mais pas par le TC- shell), elle est totalement cohérente avec la forme symbolique de la commande chmod, et devrait être préférée à la forme octale, considérée comme désuète.

Exemple

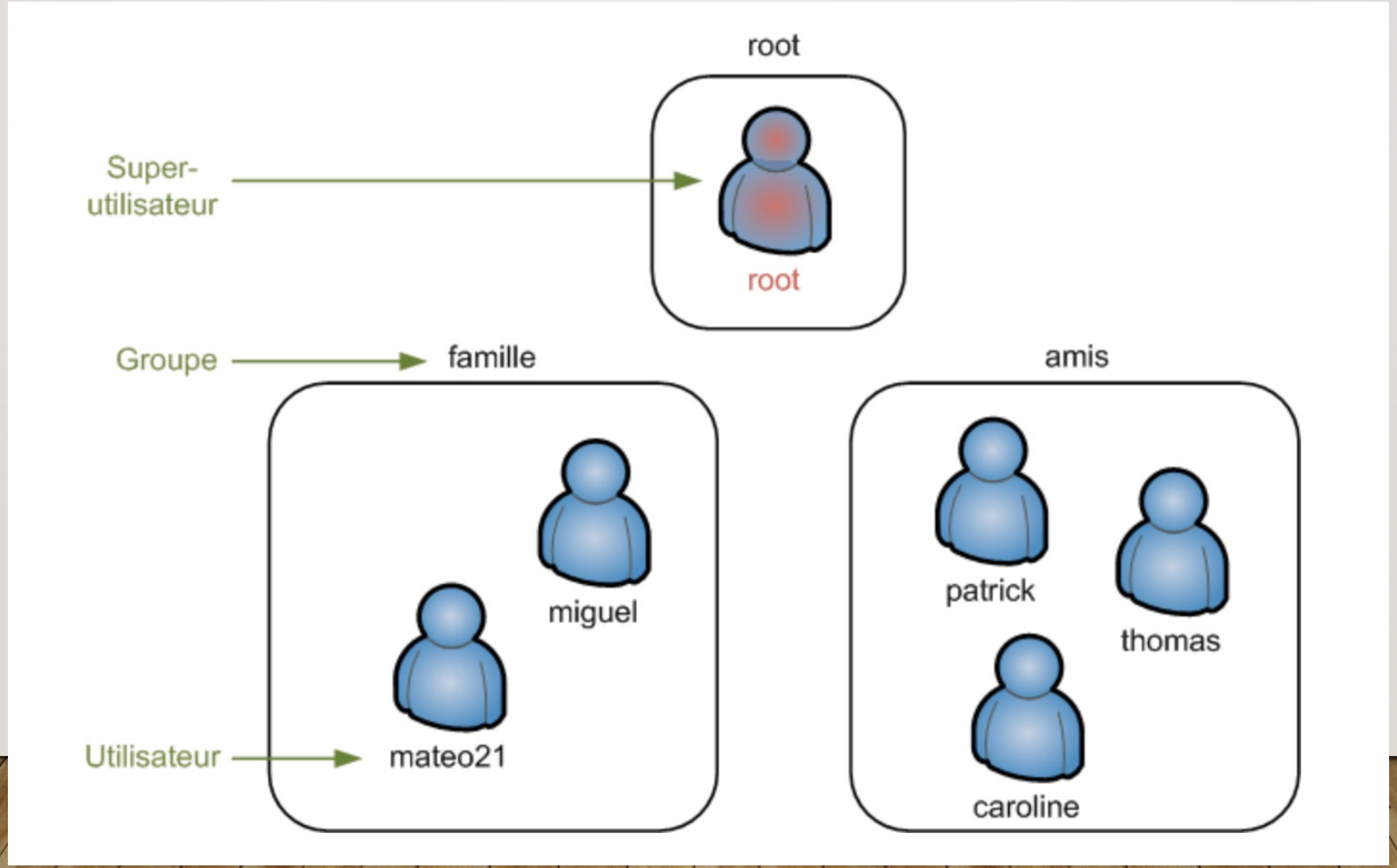
```
umask u=rwx,g=rx,o=rx
umask
0022
umask -S
u rwx,g rx,o rx
```

GESTION DES UTILISATEURS

LES UTILISATEURS ET LES DROITS

- Linux est un système multi-utilisateurs. Cela signifie que plusieurs personnes peuvent travailler simultanément sur le même OS, en s'y connectant à distance notamment.
- Puisque plusieurs utilisateurs peuvent être connectés à Linux en même temps, celui-ci doit avoir une excellente organisation dès le départ. Ainsi chaque personne a **son** propre compte utilisateur, et il existe un ensemble de règles qui disent qui a le droit de faire quoi.

L'ORGANISATION DES UTILISATEURS SOUS LINUX



ADDUSER : AJOUTER UN UTILISATEUR

- La commande adduser permet d'ajouter un utilisateur. Vous devez au minimum fournir un paramètre : le nom de l'utilisateur à créer.
  Par exemple, pour créer un compte pour Patrick :

```
root@mateo21-desktop:/home# adduser patrick
Ajout de l'utilisateur « patrick »...
Ajout du nouveau groupe « patrick » (1001)...
Ajout du nouvel utilisateur « patrick » (1001) avec le groupe « patrick »...
Création du répertoire personnel « /home/patrick »...
Copie des fichiers depuis « /etc/skel »...
```

PASSWD : CHANGER LE MOT DE PASSE

- S'il était nécessaire de changer le mot de passe de patrick par la suite, utilisez la commande passwd en indiquant en paramètre le nom du compte à modifier.

```
root@mateo21-desktop:/home# passwd patrick
Entrez le nouveau mot de passe UNIX :
Retapez le nouveau mot de passe UNIX :
passwd : le mot de passe a été mis à jour avec succès
```

Attention ! Si vous appelez `passwd` sans préciser de compte en paramètre, c'est le mot de passe de l'utilisateur sous lequel vous êtes connecté que vous changerez ! Ainsi, si vous êtes en root, c'est le mot de passe de root qui sera modifié.

DELUSER : SUPPRIMER UN COMPTE

```
deluser patrick
```

Aucune confirmation ne vous sera demandée !

✗

Ne supprimez en aucun cas votre compte utilisateur ! Par exemple, je ne dois surtout pas supprimer le compte mateo21.
En effet, si je le fais, il n'y aura plus que root sur la machine... et Ubuntu interdit de se `logger` en root. Par conséquent, au prochain démarrage de la machine vous ne pourrez pas vous connecter... et vous serez complètement coincés !

Toutefois, cette commande seule ne supprime pas le répertoire personnel de patrick. Si vous voulez supprimer aussi son `home` et tous ses fichiers personnels, utilisez le paramètre `--remove-home` :

```
deluser --remove-home patrick
```



`adduser` et `deluser` sont des commandes qui n'existent que sous Debian et tous ses descendants, dont Ubuntu.
Partout ailleurs on doit utiliser `useradd` et `userdel`, qui sont les commandes Unix traditionnelles fonctionnant partout. Elles font globalement la même chose mais de manière beaucoup plus basique : si vous n'appelez pas `passwd` vous-mêmes, le compte ne sera pas activé et n'aura pas de mot de passe.

ADDGROUP : GESTION DES GROUPE

- Si vous ne définissez rien, un groupe du même nom que l'utilisateur sera automatiquement créé
- La commande addgroup crée un nouveau groupe. Vous avez juste besoin de spécifier le nom de celui-ci en paramètre :

```
root@mateo21-desktop:/home# addgroup amis
Ajout du groupe « amis » (identifiant 1002)...
Terminé.
```

USERMOD : MODIFIER UN UTILISATEUR

- La commande usermod permet d'éditer un utilisateur. Elle possède plusieurs paramètres ;
- **-l** : renomme l'utilisateur (le nom de son répertoire personnel ne sera pas changé par contre) ;
- **-g** : change de groupe.
- **-G** : affecter à un utilisateur plusieurs groupe

  Exemple : usermod -G amis,paris,collegues patrick

- Si vous voulez **ajouter** des groupes à un utilisateur (sans perdre les groupes auxquels il appartenait avant cela), utilisez -a :
  usermod -aG amis patrick

DELGROUP : SUPPRIMER UN GROUPE

Si vous voulez supprimer un groupe, c'est tout simple :

```
delgroup amis
```

`addgroup` et `delgroup` n'existent que sous Debian et ses dérivés (même remarque que pour `adduser` et `deluser`).

Les commandes « traditionnelles » qui fonctionnent partout sont `groupadd` et `groupdel` , mais elles offrent moins d'options.

CHOWN : CHANGER LE PROPRIÉTAIRE D'UN FICHIER

- La commande chown, qui doit être utilisée **en tant que root**, attend deux paramètres au moins :
- le nom du nouveau propriétaire ;
- le nom du fichier à modifier.
- Cela donne donc :

```
chown patrick rapport.txt
```

On peut voir ensuite que patrick est bien le nouveau propriétaire du fichier :

```
root@mateo21-desktop:/home/mateo21# ls -l rapport.txt
-rw-r--r-- 1 patrick mateo21 0 2007-11-15 23:14 rapport.txt
```

CHGRP : CHANGER LE GROUPE PROPRIÉTAIRE D'UN FICHIER

`chgrp` s'utilise exactement de la même manière que `chown` à la différence près qu'il affecte cette fois le groupe propriétaire d'un fichier.

```
chgrp amis rapport.txt
```

Cette commande affectera le fichier `rapport.txt` au groupe `amis` .

Un petit `ls -l` nous confirmera que `rapport.txt` appartient désormais à patrick et au groupe `amis` :

```
root@mateo21-desktop:/home/mateo21# ls -l rapport.txt
-rw-r--r-- 1 patrick amis 0 2007-11-15 23:14 rapport.txt
```

CHOWN PEUT AUSSI CHANGER LE GROUPE PROPRIÉTAIRE D'UN FICHIER !

```
chown patrick:amis rapport.txt
```

Cela affectera le fichier à l'utilisateur patrick et au groupe `amis` .

Il suffit de séparer par un symbole deux points (« : ») le nom du nouvel utilisateur (à gauche) et le nom du nouveau groupe (à droite).

-R : AFFECTER RÉCURSIVEMENT LES SOUS-DOSSIERS

Très utile aussi, l'option `-R` de `chown` . Elle modifie tous les sous-dossiers et fichiers contenus dans un dossier pour y affecter un nouvel utilisateur (et un nouveau groupe si on utilise la technique du deux points que l'on vient de voir).

Par exemple, si je suis sadique et que je veux donner tout le contenu du dossier personnel de patrick à mateo21 (et au groupe mateo21), c'est très simple :

```
chown -R mateo21:mateo21 /home/patrick/
```

Résultat :

```
root@mateo21-desktop:/home# ls -l
total 24
drwx------  2 root    root    16384 2007-09-19 18:22 lost+found
drwxr-xr-x 62 mateo21 mateo21  4096 2007-11-15 23:19 mateo21
drwxr-xr-x  2 mateo21 mateo21  4096 2007-11-15 23:00 patrick
```